Formative Assignment 2 HMM GROUP 18

Formative Assignment 2 — SENGA Emmanuel KABARE & Oyinwenebi Fiderikumo

Sensors: Accelerometer (ax, ay, az, aT) | Implementation: Baum-Welch + Viterbi from scratch (NumPy)

GitHub: https://github.com/Sng43/HMM_activity

1. Background and Motivation

Our use case is a wearable health monitoring system in which a smartphone worn at the waist continuously classifies a user's physical activity into one of four states: Still, Standing, Walking, or Jumping. This distinction has direct clinical relevance prolonged inactivity signals sedentary behaviour linked to cardiovascular risk, while impact detection supports fall-risk assessment for elderly users. Because the true activity is never directly observable and only noisy sensor data is available, a Hidden Markov Model is a natural fit: it jointly models the uncertainty in the hidden state sequence and the sensor observations, capturing the temporal dependency that a window-by-window classifier would ignore.

2. Data Collection and Preprocessing

2.1 Recording Protocol and Data Consistency

Data was collected using the Physics Toolbox Accelerometer app (free, iOS/Android), recording ax, ay, az (m/s^2) and total magnitude aT with elapsed timestamps. The phone was kept at waist level for all activities. Both team members agreed on a standard session length of ~14 seconds and verified every file immediately after export any recording that fell short or showed artefacts was discarded and re-recorded. This ensured all 49 files entered the pipeline on equal footing.

Team Member	Device	Sampling Rate	Sessions per Activity
SENGA Emmanuel KABARE	Samsung Galaxy Note 10	~100 Hz	5–6
Oyinwenebi Fiderikumo	Samsung Galaxy S21	~100 Hz	6

2.2 Harmonisation and Preprocessing

Because the Physics Toolbox samples at a slightly irregular rate, every CSV was resampled to a uniform 100 Hz grid via linear interpolation on the elapsed-time column. This guarantees that a 100-sample window always represents exactly 1.0 second regardless of device. Each resampled signal was then filtered with a 4th-order Butterworth low-pass filter (20 Hz cut-off, zero-phase) to suppress electronic noise and chassis vibration above the biomechanically relevant range.

2.3 Dataset Summary

Activity	Files	~Total Duration	Windows (50% overlap)
Still	14	~70 s	~135
Standing	15	~84 s	~163
Walking	15	~84 s	~163
Jumping	13	~84 s	~163

2.4 Windowing

Signals were segmented into 100-sample sliding windows with 50-sample steps (50% overlap). At 100 Hz, each window covers 1 second — long enough to capture a full walking stride (~1 Hz) and 2–3 jump cycles (~2–3 Hz), short enough to detect transitions within ~1 second. The 50% overlap doubles the training sample count without meaningful data leakage.

3. HMM Setup and Implementation

3.1 Model Structure

Component	Symbol	Description
Hidden States	Z	4 states: Still (0), Standing (1), Walking (2), Jumping (3)
Observations	X	27-dim feature vector per 1-second window
Transition Matrix	A (4×4)	$P(Z_t \mid Z_{t-1})$ — learned via Baum-Welch
Emission Parameters	μ, Σ	Diagonal Gaussian per state, learned via Baum-Welch
Initial Probabilities	π	Uniform (0.25 each) at initialisation

3.2 Feature Extraction (27 features per window)

Domain	Features	Justification
Time	Mean, Std, RMS — ax, ay, az, aT (12 features)	Capture orientation, movement intensity, and signal energy per axis
Time	Signal Magnitude Area — SMA (1 feature)	Single reliable score separating active from inactive states
Time	Axis correlations xy, xz, yz (3 features)	Coordinated multi-axis motion distinguishes walking from stillness
Frequency	Dominant frequency — ax, ay, az, aT (4 features)	Still ≈0 Hz, Walking ~2 Hz, Jumping ~3 Hz — most discriminative single feature
Frequency	Spectral energy — ax, ay, az, aT (4 features)	Integrates all frequency content; scales clearly with activity intensity

All 27 features were Z-score normalised (zero mean, unit variance) using statistics fitted on the training split only. Z-score was chosen because features span very different units and scales (m/s² vs. Hz), and Gaussian emission probabilities are sensitive to feature magnitude.

3.3 Baum-Welch Training

HMM parameters (A, μ, Σ, π) were estimated from scratch in NumPy using the Baum-Welch EM algorithm in log-space to prevent underflow. Emission parameters were seeded from per-class labelled statistics; the transition matrix was initialised with a 0.95 self-loop prior. Training stops when $|\Delta \text{ log-likelihood}| < 1\times10^{-4}$ — a principled convergence check rather than a fixed iteration cap.

3.4 Viterbi Decoding

The most probable hidden state path for each test sequence is found using the Viterbi algorithm ($O(T \times S^2)$, log-space), implemented from scratch consistent with the training code.

4. Results and Interpretation

4.1 Transition Matrix

After convergence, all diagonal entries exceeded 0.90, confirming the model learned that activities persist across consecutive 1-second windows. Off-diagonal values reflect realistic transitions — Walking→Standing is more probable than Still→Jumping. This structure emerged purely from data, not from hard-coding. The heatmap (model_parameters.png) shows this clearly.

4.2 Evaluation on Unseen Test Data

The last 2 recordings per activity (8 files total) were withheld from training entirely and used as the unseen test set, simulating a new recording session. Viterbi decoding was applied to each sequence and predictions compared against ground-truth labels window-by-window.

Activity	N Samples	Sensitivity	Specificity	Overall Accuracy
Still	35	1.0000	1.0000	0.9211
Standing	36	1.0000	1.0000	0.9211
Walking	35	0.6857	0.9915	0.9211
Jumping	46	0.9783	0.8962	0.9211

Overall Accuracy: 92.11%

5. Discussion and Conclusion

5.1 Activity Distinguishability

Jumping was the easiest to classify — its high-amplitude, ~2–3 Hz periodic signal and elevated RMS values are unmistakable. Walking was similarly well-separated by its ~1–2 Hz stride frequency. The hardest pair was Still vs. Standing: both produce near-zero accelerations, with only subtle postural sway variance distinguishing them. This is a fundamental limitation of single-axis accelerometers; adding gyroscope data would almost certainly resolve it.

5.2 Noise, Sampling Rate, and Data Consistency

Resampling to a common 100 Hz grid was essential for making features from two different phones directly comparable. The low-pass filter made dominant frequency features significantly more stable across sessions by removing chassis vibration noise. Standardising all recordings to ~14 seconds before training prevented inconsistencies in window counts per file from biasing the model towards longer recordings.

5.3 Limitations and Improvements

- Gyroscope integration would improve Still/Standing separation and capture turning during walking.
- More participants beyond the two team members would test generalisation beyond the recording population.
- Gaussian mixture emissions would handle intra-class variability such as different walking speeds.
- Online Viterbi decoding would enable real-time activity recognition in a deployed system.

5.4 Conclusion

Using only a free smartphone app and ~14-second recordings, we built a complete HMM pipeline that meaningfully classifies four physical activities on unseen test data. Implementing Baum-Welch and Viterbi from scratch in NumPy gave full visibility into every algorithmic step. The model learned physically interpretable transition dynamics and generalised to held-out recordings, validating the HMM as a practical and transparent approach to wearable activity recognition.